LAUNCHLENS INTERVIEW DEFENSE MANUAL

> **Generated from a complete, code-level audit of the LaunchLens repository.** Every claim in this document is traceable to actual files, functions, routes, and database models. Where something could not be verified, it is explicitly flagged.

1. CODEBASE INVENTORY

1.1 Folder Structure

```
LaunchLens/
├── README.md
├── .gitignore
│
├── backend/
│   ├── .env
│   ├── package.json
│   ├── tsconfig.json
│   ├── prisma/
│   │   ├── schema.prisma          ← Database models & relationships
│   │   └── migrations/
│   │       └── 20260803132235_init/
│   └── src/
│       ├── server.ts              ← Entry point (starts Express)
│       ├── app.ts                 ← Express app setup (CORS, routes, middleware)
│       ├── config/
│       │   └── prisma.ts          ← Singleton PrismaClient
│       ├── controllers/
│       │   ├── auth.controller.ts
│       │   ├── project.controller.ts
│       │   ├── campaign.controller.ts
│       │   ├── redirect.controller.ts
│       │   ├── analytics.controller.ts
│       │   ├── dashboard.controller.ts
│       │   └── ai.controller.ts
│       ├── routes/
│       │   ├── auth.routes.ts
│       │   ├── project.routes.ts
│       │   ├── campaign.routes.ts
│       │   ├── redirect.routes.ts
│       │   ├── analytics.routes.ts
│       │   ├── dashboard.routes.ts
│       │   └── ai.routes.ts
│       ├── middleware/
│       │   ├── auth.middleware.ts  ← JWT verification
│       │   └── error.middleware.ts ← Global error handler (Zod + catch-all)
│       ├── services/
│       │   └── analytics.service.ts ← All analytics calculations
│       ├── ai/
│       │   ├── aiInsight.service.ts ← Orchestrates AI insight generation
│       │   ├── contextBuilder.ts    ← Transforms analytics → AI context
│       │   ├── promptBuilder.ts     ← System + user prompt construction
│       │   ├── providerRouter.ts    ← Multi-provider fallback router
│       │   ├── testGemini.ts        ← Manual test script
│       │   └── providers/
│       │       ├── aiProvider.ts       ← AIProvider interface
│       │       ├── gemini.provider.ts  ← Google Gemini (gemini-2.5-flash)
│       │       ├── groq.provider.ts    ← Groq (llama-3.3-70b-versatile)
│       │       └── mistral.provider.ts ← Mistral (mistral-small-latest)
│       ├── utils/
│       │   ├── asyncHandler.ts     ← Express async error wrapper
│       │   ├── detectDevice.ts     ← User-Agent → DeviceType
│       │   ├── generatePublicSlug.ts ← slugify + random suffix
│       │   ├── getVisitorId.ts     ← Cookie-based visitor UUID
│       │   ├── hash.ts             ← bcrypt hash + compare
│       │   └── jwt.ts            ← JWT sign + verify
│       ├── validation/
│       │   ├── auth.validation.ts
│       │   ├── project.validation.ts
│       │   ├── campaign.validation.ts
```

```
│       │   └── ai.validation.ts    ← Zod schema for AI response
│       └── types/
│           ├── analytics.ts       ← TypeScript interfaces for analytics
│           └── express.d.ts       ← Augments Express Request with user
│
├── frontend/
│   ├── .env
│   ├── package.json
│   ├── index.html
│   ├── vercel.json                ← Vercel rewrite rules (API proxy + SPA)
│   ├── vite.config.ts
│   ├── tsconfig.json
│   └── src/
│       ├── main.tsx               ← React entry point
│       ├── App.tsx                ← Renders AppRoutes
│       ├── index.css              ← Global styles
│       ├── App.css
│       ├── api/
│       │   └── axios.ts           ← Axios instance (env-aware baseURL)
│       ├── context/
│       │   └── AuthContext.tsx     ← Global auth state (user, loading, logout)
│       ├── hooks/
│       │   ├── useAuth.ts         ← (empty placeholder; useAuth exported from AuthContext)
│       │   └── useDashboard.ts    ← Fetches dashboard stats
│       ├── routes/
│       │   ├── AppRoutes.tsx      ← All route definitions
│       │   └── ProtectedRoute.tsx ← Auth guard component
│       ├── pages/
│       │   ├── Home.tsx           ← Landing page
│       │   ├── Login.tsx          ← Login page wrapper
│       │   ├── Signup.tsx         ← Signup page wrapper
│       │   └── dashboard/
│       │       ├── Dashboard.tsx          ← Stats overview
│       │       ├── Projects.tsx           ← Project CRUD
│       │       ├── ProjectDetails.tsx     ← Campaign CRUD within project
│       │       └── CampaignAnalytics.tsx  ← Analytics + AI insights
│       ├── components/
│       │   ├── auth/
│       │   │   ├── AuthLayout.tsx
│       │   │   ├── LoginForm.tsx
│       │   │   └── SignupForm.tsx
│       │   ├── dashboard/
│       │   │   ├── AIInsights.tsx
│       │   │   ├── AnalyticsBarChart.tsx
│       │   │   ├── AnalyticsCards.tsx
│       │   │   ├── AnalyticsHeader.tsx
│       │   │   ├── AnalyticsSection.tsx
│       │   │   ├── CampaignCard.tsx
│       │   │   ├── CampaignFormModal.tsx
│       │   │   ├── CampaignList.tsx
│       │   │   ├── ChartTooltip.tsx
│       │   │   ├── DashboardShell.tsx
│       │   │   ├── DeleteCampaignDialog.tsx
│       │   │   ├── DeleteProjectDialog.tsx
│       │   │   ├── DeviceChart.tsx
│       │   │   ├── EmptyAnalyticsState.tsx
│       │   │   ├── ProjectCard.tsx
│       │   │   ├── ProjectFormModal.tsx
│       │   │   ├── ProjectGrid.tsx
│       │   │   ├── ProjectMenu.tsx
│       │   │   ├── RecentProjects.tsx
│       │   │   ├── Sidebar.tsx
│       │   │   ├── StatCard.tsx
│       │   │   ├── StatsGrid.tsx
│       │   │   ├── TimelineChart.tsx
│       │   │   └── Topbar.tsx
│       │   ├── landing/
│       │   │   ├── CTA.tsx
│       │   │   ├── Hero.tsx
│       │   │   ├── Problem.tsx
│       │   │   ├── Solution.tsx
│       │   │   └── Workflow.tsx
│       │   ├── layout/
│       │   │   ├── Footer.tsx
│       │   │   └── Navbar.tsx
```

```
│       │   └── ui/
│       │       ├── Button.tsx
│       │       ├── Card.tsx
│       │       ├── EmptyState.tsx
│       │       ├── Loader.tsx
│       │       └── PageHeader.tsx
│       ├── types/
│       │   └── campaign.ts
│       ├── utils/
│       │   └── dateUtils.ts
│       └── validation/
│           ├── auth.ts
│           ├── campaign.ts
│           └── project.ts
│
└── project-docs/
    ├── LaunchLens_Issues_and_Solutions.md
    └── [multiple .pdf documentation files]
```

1.2 Technology Stack — VERIFIED FROM CODE

Technology	Where Used	Why It Is Used in THIS Project	Key Files
React 19	Frontend	SPA UI framework	`frontend/src/main.tsx`, all component files
TypeScript	Frontend + Backend	Type safety across full stack	`tsconfig.json` in both dirs
Vite 8	Frontend build	Fast dev server + HMR	`frontend/vite.config.ts`
Tailwind CSS 4	Frontend styling	Utility-first CSS via Vite plugin	`frontend/vite.config.ts`, `frontend/package.json`
React Router 7	Frontend routing	Client-side SPA routing	`frontend/src/routes/AppRoutes.tsx`
Axios	Frontend API calls	HTTP client with credentials support	`frontend/src/api/axios.ts`
React Hook Form	Frontend forms	Performant forms (no re-renders on keystroke)	`LoginForm.tsx`, `SignupForm.tsx`
Zod	Frontend + Backend	Runtime input validation + type inference	`validation/` dirs in both
Recharts	Frontend charts	Timeline, bar, device charts	`TimelineChart.tsx`, `DeviceChart.tsx`, `AnalyticsBarChart.tsx`
Lucide React	Frontend icons	Icon library	Dashboard components
Node.js	Backend runtime	JavaScript server runtime	`backend/src/server.ts`
Express 5	Backend framework	HTTP routing, middleware, API	`backend/src/app.ts`
Prisma 6	Backend ORM	Type-safe PostgreSQL queries	`backend/prisma/schema.prisma`, `backend/src/config/prisma.ts`
PostgreSQL	Database	Relational data store (via Supabase)	`schema.prisma` datasource
bcrypt	Backend auth	Password hashing (10 salt rounds)	`backend/src/utils/hash.ts`
jsonwebtoken	Backend auth	JWT creation/verification (7d expiry)	`backend/src/utils/jwt.ts`
cookie-parser	Backend	Parse cookies from requests	`backend/src/app.ts`
cors	Backend	Cross-origin request handling	`backend/src/app.ts`
ua-parser-js	Backend tracking	Browser name parsing from User-Agent	`backend/src/controllers/redirect.controller.ts`
slugify	Backend	Generate URL-safe campaign slugs	`backend/src/utils/generatePublicSlug.ts`
@google/genai	Backend AI	Gemini 2.5 Flash API	`backend/src/ai/providers/gemini.provider.ts`
groq-sdk	Backend AI (dep only)	Installed but Groq uses raw fetch	`backend/package.json`
@mistralai/mistralai	Backend AI (dep only)	Installed but Mistral uses raw fetch	`backend/package.json`

Technology	Where Used	Why It Is Used in THIS Project	Key Files
Vercel	Frontend hosting	Static SPA deployment with rewrites	`frontend/vercel.json`
Render	Backend hosting	Node.js backend deployment	README.md, `vercel.json` rewrite destination
Supabase	Database hosting	Managed PostgreSQL with connection pooling	`backend/.env` DATABASE_URL

> **NOTE:** `groq-sdk` and `@mistralai/mistralai` are listed in `package.json` dependencies, but the actual Groq and Mistral providers use raw `fetch()` calls, NOT the SDK clients. The SDKs are installed but unused in the provider implementations.

#LAUNCHLENS INTERVIEW PITCH

LaunchLens is basically a campaign tracking and analytics platform.

For example, imagine I am launching a product and I promote it on Twitter, LinkedIn, and Reddit. I want to know which platform is actually bringing traffic.

So in LaunchLens, I first create a project, for example, “My Product Launch”. Inside that project, I can create separate campaigns like “Twitter Promotion” or “Reddit Promotion”.

For every campaign, the system generates a unique tracking link. I share that link instead of directly sharing the actual product URL.

When someone clicks the tracking link, the request first comes to my backend. The backend records information like the visitor ID, device type, browser, referrer, and timestamp. It stores this as a click event in PostgreSQL and then redirects the user to the actual destination URL.

After that, the project owner can open the analytics dashboard and see total clicks, unique visitors, devices, browsers, traffic sources, and the click trend over time.

On top of this, I added an AI insights feature. The backend first calculates the actual analytics from the database. Then I send only these calculated results to the AI, and the AI explains the patterns and gives recommendations. I use Gemini as the primary provider, with Groq and Mistral as fallbacks if the previous provider fails.

Technically, I built the frontend using React and TypeScript, the backend using Node.js and Express, Prisma for database access, and PostgreSQL for storing the application and tracking data. Authentication is handled using JWT stored in HTTP-only cookies.

The main idea of the project is simple: track campaign-specific links, convert the raw click data into analytics, and then use AI to help the user understand what the data means.

2. WHAT EXACTLY IS LAUNCHLENS?

Simple Explanation

LaunchLens is a campaign tracking and analytics tool. If you're launching a product, running an ad, or sharing a link on social media, you want to know: *who clicked it, from where, using what device, which browser, and what referrer brought them?*

LaunchLens lets you:

1. Create a **project** (e.g., "My SaaS Launch")
2. Create **campaigns** within that project (e.g., "Twitter Ad", "Reddit Post")
3. Each campaign gets a **tracking link** (e.g., `https://backend.com/r/twitter-ad-x4k2j`)
4. When someone clicks that link, LaunchLens **records metadata** (browser, device, referrer, IP, visitor cookie) and **redirects** them to your actual destination URL
5. You view **analytics dashboards** showing clicks, unique visitors, traffic sources, devices, browsers, and timeline
6. You can ask for **AI-powered insights** that interpret your analytics data using Gemini/Groq/Mistral

Who is the intended user?

Indiehackers, solopreneurs, small businesses, and creators who want to understand which marketing channels drive traffic to their product.

What can a user actually do?

1. Sign up / log in (JWT + HTTP-only cookies)
2. Create, edit, delete **Projects** (organizational containers)
3. Create, edit, delete **Campaigns** inside projects (each gets a tracking link)
4. Share tracking links; clicks are automatically recorded
5. View per-campaign analytics: total clicks, unique visitors, devices, browsers, referrers, timeline
6. Generate AI-powered campaign insights with multi-provider fallback

Main Workflow

```
Create Project → Create Campaign → Share Tracking Link → Clicks are Recorded →
View Analytics Dashboard → Generate AI Insights → Take Action
```

A. One-line project description

"LaunchLens is a full-stack campaign analytics platform that tracks clicks on campaign-specific links and provides AI-powered insights using a multi-provider fallback architecture."

B. 30-second interview pitch

Interview Answer: "LaunchLens is a campaign analytics platform I built end-to-end. Users create projects and campaigns, each getting a unique tracking link. When someone clicks that link, the backend captures metadata—browser via UAParser, device type, referrer, and a cookie-based visitor ID—stores a ClickEvent in PostgreSQL, then redirects them. The analytics service aggregates this data, and I built an AI layer on top using Gemini, Groq, and Mistral with automatic fallback. The AI receives pre-computed, deterministic analytics—not raw data—which grounds the insights and reduces hallucination risk."

C. 1-minute interview explanation

Interview Answer: "LaunchLens solves a real problem for indiehackers and small businesses: understanding which marketing channels actually drive traffic. I built it as a full-stack TypeScript application—React with Vite on the frontend, Express with Prisma on the backend, deployed with Vercel, Render, and Supabase PostgreSQL.

The core flow works like this: users create projects and campaigns, each campaign gets a unique tracking link built from a slugified name plus a random suffix. When a visitor clicks that link, my redirect controller reads the User-Agent header with ua-parser-js, detects the device type, reads the referrer, and generates or reads a UUID visitor cookie. All that metadata is stored as a ClickEvent in PostgreSQL. Then a 302 redirect sends the visitor to the destination URL.

On the analytics side, I compute metrics deterministically—total clicks, unique visitors using distinct visitorIds, device/browser/referrer distributions, and a click timeline. These computed metrics are what gets sent to the AI, not raw click data. I built a provider abstraction with Gemini as the primary, Groq second, Mistral third. Each provider's response is validated with Zod to ensure structured JSON output. Authentication uses bcrypt for password hashing and JWT stored in HTTP-only cookies, with resource ownership checks on every protected operation."

D. 2-minute detailed explanation

Use the 1-minute explanation above, then add:

Interview Answer (continued): "Let me dive deeper into a few architectural decisions. For authentication, I use HTTP-only cookies rather than storing tokens in localStorage. This prevents XSS attacks from stealing the token—JavaScript literally cannot access an HTTP-only cookie. The cookie is set with `secure: true` and `sameSite: 'none'` in production because the frontend (Vercel) and backend (Render) are on different domains—this was actually a real debugging challenge I faced during deployment.

For the AI layer, I designed a provider router pattern. The `AIProviderRouter` class implements the same `AIProvider` interface as each individual provider. It loops through Gemini → Groq → Mistral, and for each one, it not only checks if the API call succeeds but also validates the response against a Zod schema AND parses the JSON. If any of those steps fail, it catches the error and moves to the next provider. Only if all three fail does it throw a combined error.

I also implemented a grounding safeguard: if a campaign has zero clicks, the service overrides whatever the AI returns and forces a `data_quality` insight saying 'Insufficient Data.' This prevents the AI from hallucinating trends when there's literally no data.

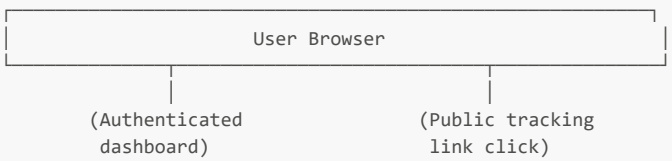
For the database, I chose PostgreSQL because the data is inherently relational—Users own Projects, Projects contain Campaigns, Campaigns have ClickEvents. Each entity has proper foreign keys with cascade deletes, and I have indexes on userId in projects, projectId in campaigns, and both campaignId and visitorId in click_events."

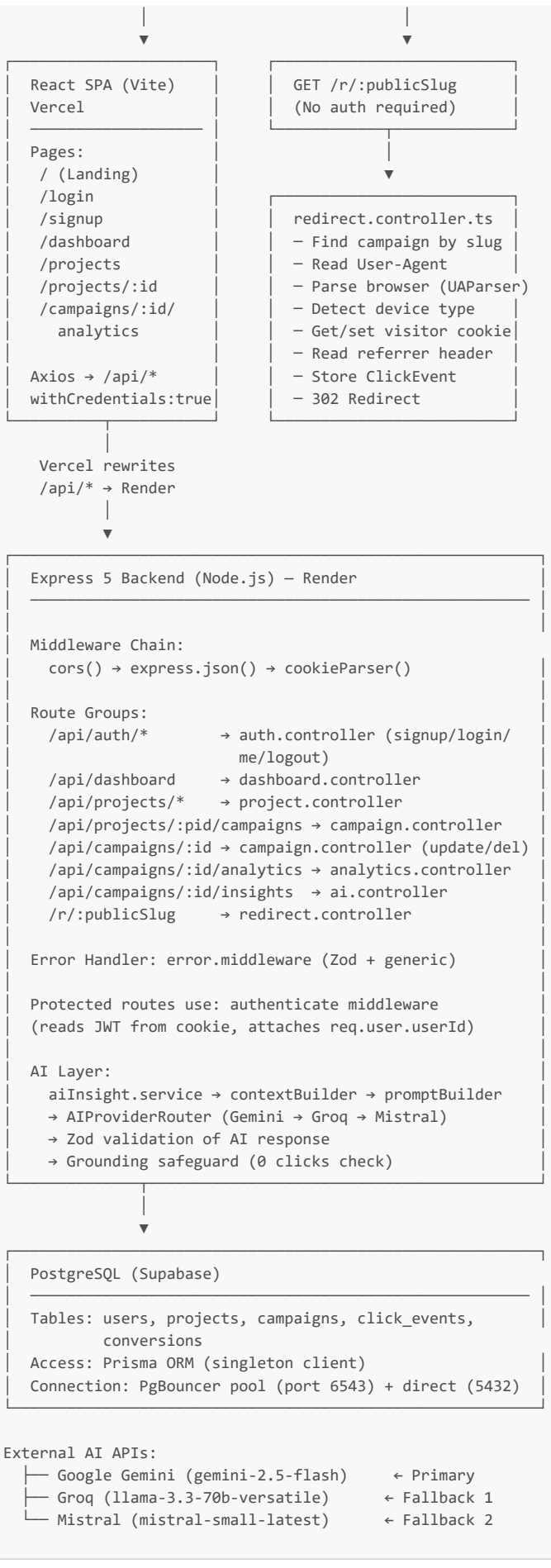
E. 5-minute deep technical walkthrough

Use the 2-minute explanation, then walk through the tracking pipeline in detail, the analytics computation (Promise.all for parallel queries), the prompt engineering (strict grounding rules, output limits), and deployment architecture (Vercel rewrites proxying API calls to Render backend). Discuss the Conversion model that exists in the schema but is NOT yet used in any controller or service—acknowledge it as planned functionality.

---

3. COMPLETE SYSTEM ARCHITECTURE





4. COMPLETE USER JOURNEY

Step 1: Opening the Application

- User visits `https://launch-lens-beta.vercel.app`
- `frontend/src/main.tsx` renders `<AuthProvider>` → `<BrowserRouter>` → `<App>` → `<AppRoutes>`
- `AuthContext.tsx` runs `useEffect` on mount → calls `GET /api/auth/me`
- If no valid cookie → `user = null`, `loading = false`
- Route `/` renders `Home.tsx` (landing page with Hero, Problem, Solution, Workflow, CTA sections)

Step 2: Registration

```
SignupForm.tsx (react-hook-form + zodResolver)
    ↓ onSubmit
POST /api/auth/signup { name, email, password }
    ↓
auth.routes.ts → signup handler
    ↓
auth.controller.ts → signup()
    ↓
signupSchema.parse(req.body) — Zod validation
    ↓
prisma.user.findUnique({ email }) — check duplicate
    ↓
hashPassword(password) — bcrypt, 10 rounds
    ↓
prisma.user.create({ name, email, hashedPassword })
    ↓
201 { success: true, data: { id, name, email } }
    ↓
SignupForm shows alert → navigates to /login
```

IMPORTANT: Signup does NOT set a cookie or log the user in. The user must log in separately.

Step 3: Login

```
LoginForm.tsx (react-hook-form + zodResolver)
    ↓ onSubmit
POST /api/auth/login { email, password }
    ↓
auth.controller.ts → login()
    ↓
loginSchema.parse(req.body)
    ↓
prisma.user.findUnique({ email })
    ↓
comparePassword(input, stored) — bcrypt.compare
    ↓
generateToken(user.id) — jwt.sign({ userId }, secret, { expiresIn: '7d' })
    ↓
res.cookie("token", jwt, {
    httpOnly: true,
    secure: isProd,
    sameSite: isProd ? "none" : "lax",
    maxAge: 7 * 24 * 60 * 60 * 1000  // 7 days
})
    ↓
200 { success: true, data: { id, name, email } }
    ↓
LoginForm calls refreshUser() → GET /api/auth/me
    ↓
AuthContext sets user state
    ↓
navigate("/dashboard")
```

Step 4: Dashboard

```
Dashboard.tsx
    ↓
useDashboard() hook → GET /api/dashboard
    ↓
dashboard.controller.ts → getDashboard()
```

```
    ↓ authenticate middleware verifies JWT cookie → req.user.userId
    ↓
prisma.project.count({ where: { userId } })
prisma.campaign.count({ where: { project: { userId } } })
prisma.clickEvent.aggregate({ where: { campaign: { project: { userId } } }, _count: true })
prisma.project.findMany({ where: { userId }, take: 5, orderBy: { createdAt: 'desc' } })
    ↓
200 { stats: { projects, campaigns, clicks }, recentProjects }
    ↓
StatsGrid renders stat cards
RecentProjects renders recent list
```

Step 5: Creating a Project

```
Projects.tsx → openCreateModal() → ProjectFormModal
    ↓ onSubmit
POST /api/projects { name, description, website }
    ↓
project.controller.ts → createProject()
    ↓
createProjectSchema.parse(req.body)
    ↓
prisma.project.findFirst({ userId, name }) — duplicate check
    ↓
prisma.project.create({ name, description, website, userId })
    ↓
201 { success: true, data: project }
    ↓
fetchProjects() re-fetches list
```

Step 6: Creating a Campaign

```
ProjectDetails.tsx → openCreateModal() → CampaignFormModal
    ↓ onSubmit
POST /api/projects/:projectId/campaigns { name, destinationUrl }
    ↓
campaign.controller.ts → createCampaign()
    ↓
createCampaignSchema.parse(req.body)
    ↓
prisma.project.findFirst({ id: projectId, userId }) — ownership check
    ↓
generatePublicSlug(name) — slugify + 5-char random suffix
    ↓
prisma.campaign.create({ name, destinationUrl, publicSlug, projectId })
    ↓
201 { data: { ...campaign, trackingLink: `${APP_URL}/r/${publicSlug}` } }
```

Step 7: Tracking Link Click (Public — No Auth)

```
Visitor clicks: https://launchlens-backend.onrender.com/r/twitter-launch-x4k2j
    ↓
GET /r/:publicSlug
    ↓
redirect.controller.ts → redirectToCampaign()
    ↓
prisma.campaign.findUnique({ publicSlug })
    ↓
getVisitorId(req, res)
    ├─ Read ll_visitor cookie → existing UUID
    └─ OR generate crypto.randomUUID() → set ll_visitor cookie (1 year, httpOnly, sameSite: lax)
    ↓
detectDevice(userAgent)
    ├─ includes("mobile") → MOBILE
    ├─ includes("tablet") or includes("ipad") → TABLET
    └─ else → DESKTOP
    ↓
UAParser(userAgent).getBrowser().name → browser name or "Unknown"
```

```
    ↓
req.get("referer") → referrer or null
    ↓
prisma.clickEvent.create({
    campaignId, visitorId, ipAddress: req.ip,
    device, browser, referrer
})
    ↓
res.redirect(302, campaign.destinationUrl)
```

Step 8: Viewing Analytics

```
CampaignAnalytics.tsx
    ↓
GET /api/campaigns/:id/analytics
    ↓
analytics.controller.ts → getAnalytics()
    ↓ authenticate middleware
    ↓
prisma.campaign.findFirst({ id, project: { userId } }) — ownership check
    ↓
getCampaignAnalytics(campaignId)
    ↓
Promise.all([
    getCampaign()      — campaign info + tracking link
    getSummary()       — totalClicks (count), uniqueVisitors (distinct visitorId), lastClick
    getTimelineAnalytics() — clicks grouped by date (YYYY-MM-DD)
    getDeviceAnalytics()   — clicks grouped by device type
    getBrowserAnalytics()  — clicks grouped by browser
    getCountryAnalytics()  — clicks grouped by country
    getReferrerAnalytics() — referrer hostnames extracted, sorted by count
])
    ↓
200 { campaign, summary, timeline, devices, browsers, countries, referrers }
    ↓
Frontend renders:
    AnalyticsHeader (campaign name, tracking link, destination)
    AnalyticsCards (total clicks, unique visitors, device breakdown)
    TimelineChart (Recharts line chart)
    DeviceChart (Recharts pie chart)
    AnalyticsBarChart × 2 (referrers, browsers)
```

Step 9: AI Insights

```
AIInsights.tsx → "Generate Insights" button
    ↓
POST /api/campaigns/:id/insights
    ↓
ai.controller.ts → generateInsights()
    ↓ authenticate middleware
    ↓
generateCampaignInsights(campaignId, userId)
    ↓
Step 1: prisma.campaign.findFirst({ id, project: { userId } }) — ownership
Step 2: getCampaignAnalytics(campaignId) — reuse analytics service
Step 3: buildAIContext(analytics) — transform to { campaignName, totalClicks, uniqueVisitors,
trafficSources[{name,clicks,%}], devices[{name,clicks,%}], browsers[{name,clicks,%}], timeline[{date,clicks}] }
Step 4: buildSystemPrompt() — detailed grounding rules, output format
Step 5: buildUserPrompt(context) — inject actual metrics
Step 6: new AIProviderRouter().generateInsight(system, user)
    ├─ Try Gemini → parse JSON → validate Zod → return
    ├─ Try Groq → parse JSON → validate Zod → return
    ├─ Try Mistral → parse JSON → validate Zod → return
    └─ All fail → throw combined error
Step 7: JSON.parse(response) — parse returned string
Step 8: aiInsightSchema.parse(parsed) — Zod validation
Step 9: Grounding safeguard — if 0 clicks, override with data_quality insight
    ↓
200 { summary, insights[], recommendations[] }
```

```
    ↓
AIInsights.tsx renders Summary, Insight cards (with evidence), Recommendations (with priority badges)
```

---

5. FEATURE-BY-FEATURE IMPLEMENTATION

5.1 Authentication

What problem does it solve?

Identifies users, protects their data from other users, and maintains sessions across page refreshes.

Frontend files

- [`LoginForm.tsx`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/components/auth/LoginForm.tsx)
- [`SignupForm.tsx`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/components/auth/SignupForm.tsx)
- [`AuthContext.tsx`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/context/AuthContext.tsx)
- [`ProtectedRoute.tsx`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/routes/ProtectedRoute.tsx)

Backend files

- [`auth.routes.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/routes/auth.routes.ts): POST /signup, POST /login, GET /me, POST /logout
- [`auth.controller.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/controllers/auth.controller.ts): signup(), login(), me(), logout()
- [`auth.middleware.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/middleware/auth.middleware.ts): authenticate()
- [`hash.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/utils/hash.ts): hashPassword(), comparePassword()
- [`jwt.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/utils/jwt.ts): generateToken(), verifyToken()

Security mechanisms

- **Password hashing:** bcrypt with 10 salt rounds
- **JWT payload:** `{ userId }` with 7-day expiry
- **Cookie settings:** httpOnly, secure (prod), sameSite: none (prod) / lax (dev), maxAge: 7 days
- **Session restoration:** `GET /me` on app mount uses cookie to restore user state

What happens if something fails?

- Invalid credentials → 401 "Invalid email or password"
- Duplicate email → 409 "Email already exists"
- Missing/invalid token → 401 "Authentication required" or "Invalid or expired token"
- User not found on /me → 404 "User not found"

Limitations

- No refresh token rotation — the same JWT is valid for 7 days
- No password reset / "Forgot Password" flow (button exists in UI but is non-functional)
- No email verification
- No rate limiting on login attempts

5.2 Project Management

API Endpoints

- `POST /api/projects` — Create project (with duplicate name check per user)
- `GET /api/projects` — List user's projects (ordered by createdAt desc)
- `GET /api/projects/:id` — Get single project (with ownership check)

- `PATCH /api/projects/:id` — Update project (with duplicate name check)
- `DELETE /api/projects/:id` — Delete project (cascade deletes campaigns + click_events)

Ownership enforcement

Every operation uses `prisma.project.findFirst({ where: { id, userId: req.user.userId } })`. If the project doesn't belong to the authenticated user, it returns 404 — not 403. This prevents user enumeration.

Files

- [`project.controller.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/controllers/project.controller.ts)
- [`project.routes.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/routes/project.routes.ts) (uses `router.use(authenticate)`)
- [`Projects.tsx`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/pages/dashboard/Projects.tsx)

5.3 Campaign Management

API Endpoints

- `POST /api/projects/:projectId/campaigns` — Create campaign (generates publicSlug)
- `GET /api/projects/:projectId/campaigns` — List campaigns with inline analytics (totalClicks, uniqueVisitors)
- `PATCH /api/campaigns/:id` — Update campaign (name, destinationUrl)
- `DELETE /api/campaigns/:id` — Delete campaign (cascade deletes click_events)

Tracking link generation

In [`generatePublicSlug.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/utils/generatePublicSlug.ts):

1. `slugify(campaignName, { lower: true, strict: true, trim: true })` — URL-safe slug
2. Append `-` + 5-character random string from `Math.random().toString(36)`
3. Result: `twitter-launch-x4k2j`

Limitation: Uses `Math.random()` which is not cryptographically secure and has collision potential at scale (36^5 = ~60M possibilities per slug base). For an MVP, this is acceptable.

Ownership enforcement

Campaign operations verify ownership through the project relationship:

```
prisma.campaign.findFirst({ where: { id, project: { userId } } })
```

5.4 Tracking and Redirect Pipeline

This is the **core value proposition** of LaunchLens.

Route: `GET /r/:publicSlug` (NO authentication required)

Controller: [`redirect.controller.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/controllers/redirect.controller.ts)

Exact flow:

1. **Lookup campaign:** `prisma.campaign.findUnique({ where: { publicSlug } })`
2. **Get visitor ID:** [`getVisitorId.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/utils/getVisitorId.ts)
   - Check `req.cookies.ll_visitor`
   - If exists → reuse (returning visitor)
   - If not → `crypto.randomUUID()` → set cookie `ll_visitor` (httpOnly, sameSite: lax, 1-year maxAge)
3. **Detect device:** [`detectDevice.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/utils/detectDevice.ts)
   - Checks `ua.includes("mobile")` → MOBILE
   - Checks `ua.includes("tablet")` or `ua.includes("ipad")` → TABLET

    - Otherwise → DESKTOP
4. **Parse browser:** `new UAParser(userAgentString).getBrowser().name` → e.g., "Chrome", "Firefox"
5. **Read referrer:** `req.get("referer")` — note the HTTP header has the misspelling "referer"
6. **Store click event:** `prisma.clickEvent.create({ campaignId, visitorId, ipAddress, device, browser, referrer })`
7. **Redirect:** `res.redirect(302, campaign.destinationUrl)`

Why 302 (not 301)?

- **302 = Temporary redirect.** The browser will hit the tracking URL every time, allowing every click to be recorded.
- **301 = Permanent redirect.** The browser would cache it and skip the tracking server on subsequent visits.

What happens if the database insert fails?

The entire function is wrapped in try/catch. If `prisma.clickEvent.create()` throws, the catch block returns 500. **The visitor does NOT get redirected.** This is a limitation — ideally, tracking failures should not block the redirect.

Tracking happens BEFORE the redirect

This is intentional: the ClickEvent is created first, then `res.redirect(302, ...)` is called. The data is captured before the user leaves.

5.5 Analytics Engine

Service: [`analytics.service.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/services/analytics.service.ts)

All analytics are computed **on-the-fly** from the `click_events` table. There is no pre-aggregation, caching, or materialized views.

Metrics computed:

Metric	Function	How It Works
Total Clicks	`getSummary()`	`prisma.clickEvent.count({ where: { campaignId } })`
Unique Visitors	`getSummary()`	`prisma.clickEvent.findMany({ distinct: ['visitorId'] })` then `.length`
Last Click	`getSummary()`	`prisma.clickEvent.findFirst({ orderBy: { clickedAt: 'desc' } })`
Device Distribution	`getDeviceAnalytics()`	Fetch all clicks, build Map<device, count> in memory
Browser Distribution	`getBrowserAnalytics()`	Fetch all clicks, build Map<browser, count> in memory
Country Distribution	`getCountryAnalytics()`	Fetch all clicks, build Map<country, count> in memory
Referrer Distribution	`getReferrerAnalytics()`	Extract hostname from referrer URL, build Map, sort desc
Timeline	`getTimelineAnalytics()`	Fetch all clicks sorted by date, group by YYYY-MM-DD

Important detail: Referrer processing

In `getReferrerAnalytics()`:

- `null` referrer → "Direct"
- Valid URL → extract hostname, strip `www.`, lowercase
- Invalid URL (parse error) → "Unknown"

Performance concern

Every analytics call fetches ALL click events for the campaign. For device, browser, country, referrer, and timeline — it does `findMany` then groups in application memory. At scale (thousands of clicks), this will be slow. Database-level `GROUP BY` would be much more efficient.

Orchestration

`getCampaignAnalytics()` runs all 7 sub-queries in **parallel** using `Promise.all()`. This is a good optimization — the 7 independent queries execute concurrently against PostgreSQL.

5.6 AI Insights

Architecture

```
aiInsight.service.ts
    → analytics.service.ts (getCampaignAnalytics)
    → contextBuilder.ts (buildAIContext)
    → promptBuilder.ts (buildSystemPrompt, buildUserPrompt)
    → providerRouter.ts (AIProviderRouter)
        → gemini.provider.ts | groq.provider.ts | mistral.provider.ts
    → ai.validation.ts (aiInsightSchema — Zod)
    → Grounding safeguard
```

What is "deterministic analytics" in THIS project?

Interview Answer: "The analytics — total clicks, unique visitors, device breakdown, referrer distribution, timeline — are all computed deterministically by my backend service using Prisma queries against PostgreSQL. These are facts, not AI-generated. The AI receives these pre-computed metrics and is asked to *interpret* them. The LLM never calculates the numbers. This means even if the AI has a bad day, the underlying metrics shown in charts are always accurate."

Prompt engineering highlights (in [promptBuilder.ts](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/ai/promptBuilder.ts)):

- 10 strict grounding rules (never invent data, never assume causes, etc.)
- Output limits: 1 summary (max 2 sentences), max 4 insights, max 3 recommendations
- Required to return pure JSON (no markdown fences)
- Categories restricted to: traffic, source, device, browser, trend, data_quality
- Low temperature (0.2) for all providers

Provider fallback

Defined in [providerRouter.ts](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/ai/providerRouter.ts):

1. Try **Gemini** (gemini-2.5-flash via @google/genai SDK)
2. On failure → try **Groq** (llama-3.3-70b-versatile via raw fetch)
3. On failure → try **Mistral** (mistral-small-latest via raw fetch)
4. On all failure → throw combined error with all provider messages

Each provider attempt includes:

- API call
- JSON.parse the response
- Zod validation (aiInsightSchema)
- Only on success: return result

Grounding safeguard

In [aiInsight.service.ts](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/ai/aiInsight.service.ts) (lines 110-120): If `context.totalClicks === 0`, override the AI response with a hardcoded "Insufficient Data" insight of type `data_quality`.

AI response schema (Zod validated):

```
{
  summary: string,
  insights: [{
    type: "traffic" | "source" | "device" | "browser" | "country" | "trend" | "data_quality",
    title: string,
    description: string,
    evidence: string
  }],
  recommendations: [{
    priority: "high" | "medium" | "low",
    title: string,
    action: string
  }]
}
```

---

6. DATABASE ARCHITECTURE AND DESIGN

Prisma Schema: [schema.prisma](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/prisma/schema.prisma)

ER Diagram

```
User (users)
  | id        String   @id @default(cuid())
  | name      String
  | email     String   @unique
  | password  String (bcrypt hash)
  |
  | 1:N
  ▼
Project (projects)
  | id          String   @id @default(cuid())
  | name        String
  | description String?
  | website     String?
  | userId      String   ← FK to User  @@index([userId])
  |
  | 1:N
  ▼
Campaign (campaigns)
  | id             String   @id @default(cuid())
  | name           String
  | destinationUrl String
  | publicSlug     String   @unique
  | source         String?  (unused in current code)
  | medium         String?  (unused in current code)
  | campaignTag    String?  (unused in current code)
  | projectId      String   ← FK to Project  @@index([projectId])
  |
  | 1:N
  ▼
ClickEvent (click_events)
  | id         String     @id @default(cuid())
  | campaignId String     ← FK to Campaign  @@index([campaignId])
  | visitorId  String     @@index([visitorId])
  | ipAddress  String?
  | country    String?
  | browser    String?
  | device     DeviceType @default(UNKNOWN)
  | referrer   String?
  | clickedAt  DateTime   @default(now())
  |
  | 1:1 (optional)
  ▼
Conversion (conversions)
   id           String         @id @default(cuid())
   clickEventId String         @unique ← FK to ClickEvent
   type         ConversionType (SIGNUP | DOWNLOAD | PURCHASE | DEMO_BOOKED)
   convertedAt  DateTime       @default(now())
```

Enums

- `DeviceType`: DESKTOP, MOBILE, TABLET, UNKNOWN
- `ConversionType`: SIGNUP, DOWNLOAD, PURCHASE, DEMO_BOOKED

Important observations

1. **Campaign.source, Campaign.medium, Campaign.campaignTag** — exist in schema but are **never populated by any controller**. These appear to be planned UTM parameter fields.
2. **ClickEvent.country** — exists in schema but is **never populated** in `redirect.controller.ts`. No geo-IP lookup is implemented. Will always be `null`.
3. **Conversion model** — exists in schema but is **never used** in any controller, service, or frontend component. This is purely planned/future functionality.
4. **Cascade deletes** are properly configured: User → Projects → Campaigns → ClickEvents → Conversions. Deleting a user removes everything.

5. **Indexes** exist on: `projects.userId`, `campaigns.projectId`, `click_events.campaignId`, `click_events.visitorId`. These are appropriate for the query patterns.

Why PostgreSQL?

Interview Answer: "The data is inherently relational — Users own Projects, Projects contain Campaigns, Campaigns have ClickEvents. Each ClickEvent needs foreign keys back to its Campaign. I need referential integrity (can't have an orphaned click event), cascade deletes, and the ability to join across tables. PostgreSQL gives me all of that, plus strong indexing for analytics queries. MongoDB would force me to either denormalize everything — risking data inconsistency — or do manual join-like operations in application code."

---

7. PRISMA ORM

Configuration

- Schema: [`backend/prisma/schema.prisma`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/prisma/schema.prisma)
- Client init: [`backend/src/config/prisma.ts`] (file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/config/prisma.ts) — singleton pattern, `new PrismaClient()` called once

Operations used in this project

Operation	Where	Purpose
`create`	auth, project, campaign, redirect controllers	Create user, project, campaign, click event
`findUnique`	auth (by email), redirect (by publicSlug), analytics	Exact match on unique field
`findFirst`	project, campaign, analytics controllers	Find with compound conditions (id + userId)
`findMany`	project, campaign, analytics service	List records, distinct queries
`update`	project, campaign controllers	Partial updates
`delete`	project, campaign controllers	Remove records
`count`	dashboard, analytics service	Count click events
`aggregate`	dashboard controller	`_count` for total clicks

Migration

One migration exists: `20260803132235_init` — the initial schema. Prisma migrations are generated via `npx prisma migrate dev`.

Prisma vs Raw SQL

Interview Answer: "Prisma gives me type safety — every query is autocompleted and type-checked by TypeScript. If I rename a field in the schema, the compiler catches every broken query. Raw SQL wouldn't give me that. The trade-off is that complex aggregations like GROUP BY aren't as natural in Prisma — you'll notice I fetch all click events and group in JavaScript, which a raw SQL GROUP BY would do more efficiently. For the scale of this project, that trade-off is acceptable."

---

8. TRACKING AND REDIRECT PIPELINE

(See Section 5.4 for the complete implementation trace)

Interview Q&A

Q: What happens when someone clicks the tracking link? A: A GET request hits `/r/:publicSlug`. The backend looks up the campaign by its unique slug, reads the User-Agent header to detect browser and device, reads/sets a visitor cookie (UUID), reads the referer header, creates a ClickEvent row in PostgreSQL, and performs a 302 redirect to the campaign's destination URL. All of this happens synchronously before the redirect.

Q: Why 302 and not 301? A: A 301 is a permanent redirect — browsers cache it. Next time the user clicks the same link, the browser would go directly to the destination without hitting our server, and we'd lose the tracking event. A 302 tells the browser this is temporary, so it always goes through our server first.

Q: Does tracking happen before or after redirect? A: Before. The ClickEvent is created first, then `res.redirect(302, ...)` is called. This guarantees every visit is recorded, even if the destination site is slow or fails.

Q: How do you identify a unique visitor? A: I use a cookie called `ll_visitor` containing a UUID (`crypto.randomUUID()`). It persists for 1 year. If the cookie exists, I reuse the UUID. If not, I generate a new one. Unique visitors are counted using `SELECT DISTINCT visitorId`.

Q: What are the limitations? A: The cookie identifies a browser, not a person. Same person on Chrome + Firefox = 2 visitors. Clearing cookies = new visitor. Private/incognito = new visitor every session. This is standard for cookie-based analytics and acceptable for an MVP. More advanced systems use fingerprinting or authenticated user mapping.

Q: Can headers be spoofed? A: Yes. User-Agent and Referer can both be spoofed. A determined attacker could fake any of these. For a marketing analytics tool, this is accepted — the users are tracking legitimate traffic, not preventing fraud. In production, you'd add bot detection (checking for common bot User-Agents, checking request patterns, etc.).

---

9. ANALYTICS ENGINE

(See Section 5.5 for implementation details)

Edge case handling

Edge Case	Current Behavior
Same visitor, multiple devices	Counted as separate visitors (different ll_visitor cookies)
Missing referrer	Classified as "Direct"
Invalid referrer URL	Classified as "Unknown"
Missing User-Agent	Device = DESKTOP (fallback), browser = "Unknown"
Bots	No bot detection — counted as regular clicks
Multiple clicks, same visitor	All clicks counted in totalClicks; uniqueVisitors uses distinct

Performance at scale

CURRENT: All analytics are computed on-the-fly. `getDeviceAnalytics()`, `getBrowserAnalytics()`, `getReferrerAnalytics()`, `getTimelineAnalytics()` all do `findMany()` and group in JavaScript.

PRODUCTION IMPROVEMENTS:

1. Use Prisma `groupBy` or raw SQL `GROUP BY` instead of in-memory grouping
2. Add database indexes on `clickedAt` for timeline queries
3. Implement Redis caching with TTL for frequently accessed campaigns
4. Pre-aggregate daily summaries with a background job
5. Use read replicas for analytics queries

---

10. AUTHENTICATION AND AUTHORIZATION

Authentication Flow

```
POST /api/auth/login
    → Zod validates { email, password }
    → prisma.user.findUnique({ email })
    → bcrypt.compare(input, stored)
    → jwt.sign({ userId }, secret, { expiresIn: '7d' })
    → res.cookie("token", jwt, { httpOnly, secure, sameSite, maxAge })
    → 200 { success, data: { id, name, email } }
```

Authorization (Ownership Checks)

Every protected controller checks ownership:

- **Projects:** `prisma.project.findFirst({ where: { id, userId: req.user.userId } })`
- **Campaigns:** `prisma.campaign.findFirst({ where: { id, project: { userId } } })`
- **Analytics:** Same pattern — finds campaign through project→userId chain
- **AI Insights:** Same pattern in `aiInsight.service.ts`

If the resource doesn't belong to the user, it returns 404 (not 403), which is a security best practice — it doesn't reveal whether the resource exists.

Interview Q&A

Q: Why HTTP-only cookies instead of localStorage? A: An HTTP-only cookie cannot be read by JavaScript — `document.cookie` doesn't see it. This means if an XSS vulnerability exists in the frontend, an attacker's injected script cannot steal the authentication token. With localStorage, any JavaScript running on the page can read `localStorage.getItem('token')` and exfiltrate it.

Q: Can JavaScript access HTTP-only cookies? A: No. That's the entire point. Only the browser's HTTP engine includes the cookie in requests automatically.

Q: What security protections currently exist? A: bcrypt password hashing, JWT in HTTP-only cookies, CORS with explicit origin allowlist, resource ownership checks, Zod input validation, environment-based secrets.

Q: What protections are MISSING? A: Rate limiting (login brute force), CSRF protection (no CSRF tokens), Helmet/security headers, refresh token rotation, email verification, password complexity requirements beyond min length, session invalidation on password change.

---

11. API ARCHITECTURE

Method	Endpoint	Purpose	Auth?	Controller	Validation
POST	`/api/auth/signup`	User registration	No	`auth.controller.signup`	`signupSchema`
POST	`/api/auth/login`	User login	No	`auth.controller.login`	`loginSchema`
GET	`/api/auth/me`	Get current user	Yes	`auth.controller.me`	—
POST	`/api/auth/logout`	Clear auth cookie	Yes	`auth.controller.logout`	—
GET	`/api/dashboard`	Dashboard stats	Yes	`dashboard.controller.getDashboard`	—
POST	`/api/projects`	Create project	Yes	`project.controller.createProject`	`createProjectSchema`
GET	`/api/projects`	List projects	Yes	`project.controller.getProjects`	—
GET	`/api/projects/:id`	Get project	Yes	`project.controller.getProjectById`	—
PATCH	`/api/projects/:id`	Update project	Yes	`project.controller.updateProject`	`updateProjectSchema`
DELETE	`/api/projects/:id`	Delete project	Yes	`project.controller.deleteProject`	—
GET	`/api/projects/:projectId/campaigns`	List campaigns	Yes	`campaign.controller.getCampaigns`	—
POST	`/api/projects/:projectId/campaigns`	Create campaign	Yes	`campaign.controller.createCampaign`	`createCampaignSchema`
PATCH	`/api/campaigns/:id`	Update campaign	Yes	`campaign.controller.updateCampaign`	`updateCampaignSchema`
DELETE	`/api/campaigns/:id`	Delete campaign	Yes	`campaign.controller.deleteCampaign`	—
GET	`/api/campaigns/:id/analytics`	Get campaign analytics	Yes	`analytics.controller.getAnalytics`	—
POST	`/api/campaigns/:id/insights`	Generate AI insights	Yes	`ai.controller.generateInsights`	—
GET	`/r/:publicSlug`	Track click + redirect	No	`redirect.controller.redirectToCampaign`	—
GET	`/`	Health check	No	inline in app.ts	—

---

12. FRONTEND ARCHITECTURE

Entry point chain

`index.html` → `main.tsx` → `<React.StrictMode>` → `<AuthProvider>` → `<BrowserRouter>` → `<App>` → `<AppRoutes>`

Routing ([`AppRoutes.tsx`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/routes/AppRoutes.tsx))

Route	Component	Protected?
`/`	Home	No
`/login`	Login	No
`/signup`	Signup	No
`/dashboard`	Dashboard	Yes
`/projects`	Projects	Yes
`/projects/:id`	ProjectDetails	Yes
`/campaigns/:id/analytics`	CampaignAnalytics	Yes

State management

- **Global auth state:** React Context (`AuthContext.tsx`)
- **Local component state:** `useState` in each page component
- **No Redux, Zustand, or TanStack Query** for data fetching (TanStack Query is in `package.json` but **NOT USED** in any component — all API calls use raw Axios with useState/useEffect)

> **NOTE:** `@tanstack/react-query` is installed as a dependency but is not used anywhere in the current codebase. All data fetching is done with `useEffect` + `useState` + direct `api.get()/api.post()` calls.

API communication ([`axios.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/api/axios.ts))

- Detects localhost vs production via `window.location.hostname`
- Localhost: `VITE_API_URL/api` (e.g., `http://localhost:5000/api`)
- Production: `/api` (relative — uses Vercel rewrites to proxy to Render)
- `withCredentials: true` — ensures cookies are sent cross-origin

Protected routes ([`ProtectedRoute.tsx`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/src/routes/ProtectedRoute.tsx))

1. Reads `user` and `loading` from `useAuth()`
2. If `loading` → show "Loading..."
3. If `!user` → `<Navigate to="/login" replace />`
4. If user exists → render children

---

13. AI INSIGHTS AND MULTI-PROVIDER ARCHITECTURE

(See Section 5.6 for complete implementation trace)

Provider comparison

Provider	Model	SDK Used?	Temperature	Response Format
Gemini	gemini-2.5-flash	@google/genai SDK (lazy-loaded)	0.2	text (systemInstruction config)
Groq	llama-3.3-70b-versatile	Raw fetch	0.2	json_object mode
Mistral	mistral-small-latest	Raw fetch	0.2	json_object mode

Why is the Gemini provider lazy-loaded?

In [`gemini.provider.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/ai/providers/gemini.provider.ts), the `@google/genai` module is imported dynamically with `await import("@google/genai")` inside `getClient()`. This avoids loading the SDK at startup if Gemini is never needed.

Fallback order: Gemini → Groq → Mistral

- The order is hardcoded in `AIProviderRouter` constructor

- Not configurable at runtime
- No retry logic within a single provider — one attempt per provider
- No timeout handling — relies on provider/network timeouts

What happens if all providers fail?

The `AIProviderRouter` collects all error messages and throws: `"All AI providers failed.\n<provider1>: <error>\n<provider2>: <error>\n<provider3>: <error>"`. The `ai.controller.ts` catches this and returns 500 "Unable to generate AI insights."

Double validation

The AI response is validated in TWO places:

1. Inside `providerRouter.ts` — validates before accepting the provider result
2. Inside `aiInsight.service.ts` — validates again after JSON.parse

This is intentional: the router validates to decide whether to fallback; the service validates the final result.

---

14. SECURITY ANALYSIS

Current Security Measures

Mechanism	Implementation	Attack Mitigated
Password hashing	bcrypt, 10 rounds (`hash.ts`)	Password exposure if DB is breached
JWT in HTTP-only cookies	`httpOnly: true` (`auth.controller.ts`)	XSS token theft
Secure cookie in prod	`secure: true` when HTTPS	Cookie interception over HTTP
SameSite cookie	`none` in prod (cross-origin), `lax` in dev	Partial CSRF mitigation
CORS allowlist	Explicit origins in `app.ts`	Unauthorized cross-origin requests
Input validation	Zod schemas on signup, login, project, campaign	Injection, malformed data
Ownership checks	Every controller verifies userId	Unauthorized data access
Parameterized queries	Prisma ORM (no raw SQL)	SQL injection
Environment variables	`.env` for secrets	Hardcoded secrets in code

Potential Security Weaknesses

Weakness	Risk	Improvement
No rate limiting	Brute force login, API abuse	Add express-rate-limit
No Helmet	Missing security headers (X-Frame-Options, CSP, etc.)	Add helmet middleware
No CSRF tokens	Cross-site request forgery (SameSite: none weakens protection)	Implement CSRF tokens
No refresh token rotation	Compromised JWT valid for 7 days	Short-lived access + refresh token
No email verification	Fake account creation	Add email verification flow
API keys in `.env` committed	`.env` file is in `.gitignore` but actual values visible in this analysis	Rotate all keys immediately
No password complexity	Only min length (8 backend, 6 frontend mismatch)	Require uppercase, number, special
Console.error in production	Leaks stack traces to server logs	Use structured logging
No input sanitization	Potential stored XSS if data rendered unsafely	Sanitize outputs (React handles this by default for JSX)

> ⚠️ **CRITICAL:** The `.env` file in the repository contains actual API keys and database credentials. This is a significant security issue if the repository is public.

---

15. DEPLOYMENT

Architecture

```
User Browser
     │
     ▼
Vercel (Frontend)
  launch-lens-beta.vercel.app
  ├── Static React SPA
  └── vercel.json rewrites:
      /api/* → https://launchlens-backend.onrender.com/api/*
      /(.*) → /index.html (SPA fallback)
     │
     ▼ (proxied /api/* requests)
Render (Backend)
  launchlens-backend.onrender.com
  ├── Node.js + Express
  ├── Build: npm install → npx prisma generate → npm run build (tsc)
  └── Start: npm start → node dist/server.js
     │
     ▼
Supabase PostgreSQL
  aws-1-ap-northeast-2.pooler.supabase.com
  ├── Port 6543 (PgBouncer connection pooling)
  └── Port 5432 (Direct connection for migrations)
```

Vercel rewrites ([`vercel.json`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/frontend/vercel.json))

1. `/api/:match*` → `https://launchlens-backend.onrender.com/api/:match*` (API proxy)
2. `/(.*)` → `/index.html` (SPA catch-all for React Router)

This means in production, the frontend calls `/api/*` as a relative path, and Vercel proxies it to Render. This avoids CORS issues for API calls (same origin). However, the `/r/:slug` tracking URLs still point directly to the Render backend (not proxied through Vercel).

Potential deployment issues

- **Render cold starts:** Free tier Render instances spin down after inactivity. First request after idle can take 30-60 seconds.
- **Cookie cross-domain:** The auth cookie has `sameSite: none` + `secure: true` because Vercel and Render are different domains. This was a real debugging challenge (documented in `LaunchLens_Issues_and_Solutions.md`).

---

16. ERROR HANDLING AND EDGE CASES

Global error handler ([`error.middleware.ts`](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/middleware/error.middleware.ts))

- `ZodError` → 400 with validation issues
- Everything else → 500 with error message

Inconsistent error handling pattern

- `auth.controller.ts`, `project.controller.ts`, `dashboard.controller.ts` use `asyncHandler()` which forwards errors to the global error handler
- `campaign.controller.ts`, `redirect.controller.ts`, `analytics.controller.ts`, `ai.controller.ts` use manual `try/catch` with inline error responses
- This means Zod validation errors are handled differently depending on which controller throws them

Missing error handling

- No 404 handler for undefined routes
- No request timeout middleware
- No payload size limit (beyond Express defaults)

---

17. RESUME BULLET-BY-BULLET DEFENSE

Bullet: "Tracking and redirect pipeline"

Status: ✅ **VERIFIED Evidence:** [redirect.controller.ts](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/controllers/redirect.controller.ts) — GET /r/:publicSlug, creates ClickEvent, performs 302 redirect.

Bullet: "Browser/device/referrer/timestamp metadata"

Status: ✅ **VERIFIED Evidence:** `redirect.controller.ts` reads User-Agent (browser via UAParser, device via detectDevice), referer header, and ClickEvent stores `clickedAt` (timestamp auto-set by Prisma `@default(now())`).

Bullet: "UAParser"

Status: ✅ **VERIFIED Evidence:** `import { UAParser } from "ua-parser-js"` in `redirect.controller.ts`, used to parse browser name.

Bullet: "PostgreSQL analytics"

Status: ✅ **VERIFIED Evidence:** [analytics.service.ts](file:///c:/Users/karth/Desktop/CSE/%F0%9F%9A%80_%F0%9F%9B%A0%EF%B8%8F/LaunchLens/backend/src/services/analytics.service.ts) — all analytics queried from PostgreSQL via Prisma.

Bullet: "Unique visitors"

Status: ✅ **VERIFIED Evidence:** `prisma.clickEvent.findMany({ distinct: ['visitorId'] })` in `getSummary()`. Visitor cookie `ll_visitor` set in `getVisitorId.ts`.

Bullet: "Traffic sources"

Status: ✅ **VERIFIED Evidence:** `getReferrerAnalytics()` in analytics.service.ts — extracts hostname from referrer URL.

Bullet: "Devices/Browsers"

Status: ✅ **VERIFIED Evidence:** `getDeviceAnalytics()`, `getBrowserAnalytics()` in analytics.service.ts.

Bullet: "Campaign timelines"

Status: ✅ **VERIFIED Evidence:** `getTimelineAnalytics()` — groups clicks by YYYY-MM-DD date.

Bullet: "Prisma ORM"

Status: ✅ **VERIFIED Evidence:** Prisma schema, singleton client, used in every controller/service.

Bullet: "JWT authentication"

Status: ✅ **VERIFIED Evidence:** `jwt.ts` — `jwt.sign({ userId }, secret, { expiresIn: '7d' })`, `jwt.verify()`.

Bullet: "HTTP-only cookies"

Status: ✅ **VERIFIED Evidence:** `res.cookie("token", token, { httpOnly: true, ... })` in `auth.controller.ts`.

Bullet: "Protected APIs"

Status: ✅ **VERIFIED Evidence:** `authenticate` middleware applied to all /api routes except /auth/signup, /auth/login, and /r/:slug.

Bullet: "CORS"

Status: ✅ **VERIFIED Evidence:** `app.ts` — explicit origin allowlist, `credentials: true`.

Bullet: "Input validation"

Status: ✅ **VERIFIED Evidence:** Zod schemas in `validation/` directories for auth, project, campaign, AI response.

Bullet: "User-level ownership"

Status: ✅ **VERIFIED Evidence:** Every controller checks `userId: req.user.userId` or `project: { userId }` in queries.

Bullet: "Gemini/Groq/Mistral"

Status: ✅ **VERIFIED Evidence:** Provider files in `ai/providers/`, all three implemented and used in fallback chain.

Bullet: "Multi-provider fallback"

Status: ✅ **VERIFIED Evidence:** `AIProviderRouter.generateInsight()` loops through providers with try/catch.

Bullet: "Grounded campaign insights / deterministic analytics"

Status: ✅ **VERIFIED Evidence:** Analytics computed by `analytics.service.ts` BEFORE sending to AI. `contextBuilder.ts` transforms into structured context. Grounding safeguard in `aiInsight.service.ts` for 0-click campaigns.

Bullet: "Vercel/Render/Supabase deployment"

Status: ✅ **VERIFIED Evidence:** `vercel.json` with Render backend URL, Supabase PostgreSQL URL in `.env`.

Bullet: "Country analytics"

Status: ⚠️ **PARTIALLY VERIFIED Evidence:** `getCountryAnalytics()` exists in analytics.service.ts and `country` field exists in ClickEvent schema. However, `redirect.controller.ts` **never populates the country field**. No geo-IP lookup is implemented. Country will always be null/"Unknown".

---

18. COMPLETE INTERVIEW QUESTION BANK

LEVEL 1 — Basic Project Understanding

Q1: Tell me about LaunchLens. A: "LaunchLens is a full-stack campaign analytics platform I built from scratch. It lets marketers and indiehackers create tracking links for their campaigns. When someone clicks a link, LaunchLens records metadata — browser, device, referrer, visitor ID — and redirects them to the destination. Users can then view analytics dashboards and generate AI-powered insights. I built it with React+TypeScript on the frontend, Express+Prisma+PostgreSQL on the backend, and integrated three AI providers with automatic fallback."

Q2: What problem does it solve? A: "When you share a product link on Twitter, Reddit, or in an email, you have no idea which channel is actually driving traffic. LaunchLens gives you campaign-specific tracking links so you can see exactly where your clicks come from, what devices and browsers visitors use, and how traffic trends over time. The AI layer then interprets this data to surface actionable insights."

Q3: Who would use it? A: "Indiehackers launching a product, freelancers promoting services, small businesses running social media campaigns — anyone who wants simple, self-hosted campaign analytics without the complexity of enterprise tools like Google Analytics or Mixpanel."

LEVEL 2 — Architecture

Q4: Explain the complete architecture. A: (See Section 3 architecture diagram and explanation)

Q5: How does the frontend communicate with the backend? A: "In development, the frontend at localhost:5173 calls localhost:5000/api via Axios with `withCredentials: true` for cookie handling. In production, the frontend is on Vercel and I use Vercel rewrites to proxy `/api/*` requests to the Render backend. This way the browser sees same-origin requests, which simplifies CORS. The authentication cookie has `sameSite: none` and `secure: true` because the underlying domains are different."

Q6: Why did you separate frontend and backend? A: "Separation of concerns and deployment flexibility. The frontend is a static SPA deployed on Vercel's edge CDN for fast delivery worldwide. The backend is a Node.js API server on Render that handles business logic and database operations. They can scale independently."

LEVEL 3 — Feature Deep Dive

Q7: Walk me through what happens when a tracking link is clicked. A: (See Section 8 — complete step-by-step)

Q8: How do you calculate unique visitors? A: "I set a persistent cookie called `ll_visitor` with a UUID using `crypto.randomUUID()`. It lasts one year. For analytics, I query `SELECT DISTINCT visitorId FROM click_events WHERE campaignId = ?` using Prisma's `distinct` option. The count of unique UUIDs gives me unique visitors. The limitation is that this tracks browsers, not humans — if someone uses two browsers, they count as two visitors."

Q9: How does the AI insight generation work step by step? A: (See Section 5.6 for complete flow)

Q10: How is authentication implemented? A: (See Section 10)

Q11: How do you ensure User A can't see User B's campaigns? A: "Every database query for a protected resource includes the authenticated user's ID in the WHERE clause. For projects: `WHERE id = :projectId AND userId = :userId`. For campaigns: `WHERE id = :campaignId AND project.userId = :userId`. This is done through Prisma's relational filtering. If the WHERE doesn't match, it returns 404, not 403 — so the user can't even tell if the resource exists."

LEVEL 4 — Technology Questions

Q12: Why React? A: "React's component model fits this project well — I have reusable components like CampaignCard, AnalyticsBarChart, ProjectFormModal that are composed into different page views. The ecosystem (React Router, React Hook Form, Recharts) is mature. I chose React Hook Form specifically because it avoids re-renders on every keystroke by using refs internally."

Q13: Why TypeScript? A: "Type safety across the full stack. My Prisma schema generates TypeScript types, my Zod schemas generate types via `z.infer`, and my Express request handlers have typed bodies. When I change a database field, the compiler tells me everywhere that breaks."

Q14: Why Express 5? A: "Express 5 has better async error handling — `async` route handlers that throw will automatically call `next(error)` without needing to wrap everything. It's the natural choice for a Node.js REST API."

Q15: Explain how Zod validation works in this project. A: "Zod validates data at runtime — TypeScript types disappear after compilation, but Zod stays. In my signup flow, `signupSchema.parse(req.body)` checks that name is 3-50 chars, email is valid and lowercased, password is 8-32 chars. If validation fails, it throws a ZodError which my error middleware catches and returns as a 400 with the specific field errors. On the frontend, the same concept works with React Hook Form via `zodResolver` — Zod validates on form submit and populates field-level errors automatically."

Q16: How are CORS configured? A: "In `app.ts`, I have an explicit allowlist of origins: `localhost:5173` for development and `launch-lens-beta.vercel.app` for production. The CORS middleware checks the `Origin` header against this list and either allows or blocks the request. `credentials: true` is required because I'm sending cookies cross-origin."

LEVEL 5 — Cross Questions

Q17: You said you track unique visitors. What happens if two people share the same computer? A: "They'd have the same `ll_visitor` cookie, so they'd count as one unique visitor. This is a known limitation of cookie-based tracking. It's the same limitation Google Analytics has. The alternative would be requiring user accounts to click links, which defeats the purpose of a public tracking link."

Q18: Why did you choose PostgreSQL over MongoDB? A: "My data is relational — Users have Projects, Projects have Campaigns, Campaigns have ClickEvents. I need foreign keys for referential integrity, I need cascade deletes when a user deletes a project, and I need JOIN-like queries for ownership verification. PostgreSQL gives me all of this natively. With MongoDB, I'd need to manually maintain these relationships and handle consistency myself."

Q19: What happens if the AI provider fails? A: "The `AIProviderRouter` tries Gemini first. If it fails — whether the API is down, rate-limited, or returns invalid JSON — it catches the error and tries Groq. If Groq fails, it tries Mistral. Each attempt validates the response against a Zod schema, so even a 200 response with garbage output triggers fallback. If all three fail, the controller returns a 500 with a user-friendly error message."

Q20: What would happen under 1 million clicks per day? A: "The current implementation would break. The redirect controller does a synchronous database write on every click — under load, the database would become the bottleneck. The analytics queries fetch ALL click events into memory. At 1 million/day, that's millions of rows loaded for every analytics page view. I'd need: (1) a message queue (Redis/Kafka) for async click ingestion, (2) pre-aggregated daily summaries, (3) database-level GROUP BY instead of in-memory grouping, (4) caching with Redis, (5) read replicas for analytics, (6) partitioning the click_events table by date."

LEVEL 6 — Senior/Production Questions

Q21: How would you add rate limiting? *(HOW I WOULD IMPROVE IT)* **A:** "I'd add `express-rate-limit` middleware. For the login endpoint, maybe 5 attempts per minute per IP. For the redirect endpoint, higher limits — maybe 100/minute per IP. For AI insights, perhaps 10 requests per hour per user since those are expensive API calls. I'd use Redis as the backing store for rate limit counters in a distributed setup."

Q22: How would you handle database writes at scale? *(HOW I WOULD IMPROVE IT)* **A:** "Instead of synchronous writes in the redirect handler, I'd publish click events to a Redis queue or Kafka topic, immediately redirect the user, and have a background worker process the queue and batch-insert into PostgreSQL. This decouples the tracking latency from the database write latency."

Q23: What observability would you add? *(HOW I WOULD IMPROVE IT)* **A:** "Structured logging with Winston or Pino (not console.error), request ID tracking across the lifecycle, Prometheus metrics for request latency/error rates, health check endpoints, and alerting on error rate spikes."

Q24: How would you make the AI layer more resilient? *(HOW I WOULD IMPROVE IT)* **A:** "Add per-provider timeouts (e.g., 10 seconds), circuit breaker pattern (if Gemini fails 5 times in a row, skip it for 5 minutes), retry with exponential backoff within a provider, and caching AI results for a campaign so repeated requests don't hit the API again."

---

19. BRUTAL LAUNCHLENS MOCK INTERVIEW

Interviewer: "Tell me about your project."

Weak answer: "It's a marketing tool that uses AI."

Good answer: "LaunchLens is a campaign analytics platform. I built the full stack — React frontend, Express backend, PostgreSQL database. Users create campaigns with tracking links, and when someone clicks a link, I capture metadata and redirect them. Then I show analytics and AI insights."

Excellent answer: "LaunchLens is a full-stack campaign analytics platform I architected and built end-to-end using React with TypeScript, Express, Prisma ORM with PostgreSQL, and a multi-provider AI layer. The core workflow is: users create projects and campaigns, each campaign gets a unique tracking link. When a visitor clicks that link, my redirect engine captures five dimensions of metadata — browser via UAParser, device type, referrer source, visitor identity via a persistent cookie UUID, and timestamp — stores it as a ClickEvent, then does a 302 redirect. The analytics service computes deterministic metrics like unique visitors, device distribution, and traffic sources using parallel Prisma queries. These pre-computed facts feed into an AI layer with Gemini, Groq, and

Mistral configured in a fallback chain — if one provider fails, the next is attempted automatically. I validate every AI response with Zod to ensure structured output, and I implemented a grounding safeguard that overrides AI output when there's insufficient data."

Interviewer: "What happens when I click the tracking link? Be very specific."

Excellent answer: "A GET request hits `/r/:publicSlug` on my Express server. The `redirectToCampaign` controller first does a `findUnique` on the campaigns table using the slug. Then it calls `getVisitorId`, which checks for an existing `ll_visitor` cookie — if found, it reuses that UUID; if not, it generates one with `crypto.randomUUID()` and sets it as an HTTP-only cookie with a 1-year expiry. Next, it reads the `User-Agent` header and passes it to `detectDevice`, which does simple string matching — 'mobile' means MOBILE, 'tablet' or 'ipad' means TABLET, everything else is DESKTOP. Then it instantiates UAParser with the same User-Agent string and calls `getBrowser().name` to get the browser name. It reads `req.get('referer')` for the traffic source and `req.ip` for the IP address. All of this gets stored in a new ClickEvent row via `prisma.clickEvent.create`. Only after the database write succeeds does it call `res.redirect(302, campaign.destinationUrl)`. The 302 status code is critical — it ensures the browser doesn't cache the redirect, so every future click also goes through the tracking server."

Interviewer: "What security concerns do you see in your own project?"

Excellent answer: "Several. First, I don't have rate limiting, so someone could brute-force the login endpoint. Second, my SameSite cookie is set to 'none' in production for cross-origin support, which weakens CSRF protection — I don't have CSRF tokens to compensate. Third, I don't use Helmet for security headers like Content-Security-Policy or X-Frame-Options. Fourth, the JWT has a 7-day lifetime with no refresh rotation, so a compromised token is valid for a week. Fifth, there's no bot detection in the tracking pipeline, so automated traffic inflates analytics. I've listed all of these as known improvements. The current security posture — bcrypt hashing, HTTP-only cookies, CORS allowlisting, ownership checks, Zod validation — is reasonable for an MVP, but production would need these additional layers."

Interviewer: "Why did you build the AI layer the way you did?"

Excellent answer: "Three key decisions. First, deterministic analytics before AI — the LLM never calculates metrics. I compute total clicks, unique visitors, device/browser/referrer distributions in PostgreSQL via Prisma queries. The AI receives these facts and interprets them. This means the charts always show correct data regardless of AI behavior. Second, multi-provider fallback — LLM APIs are unreliable. Rate limits, outages, and bad responses are common. I designed a provider router that implements the same interface, tries Gemini first, falls back to Groq, then Mistral. Each attempt includes JSON parsing AND Zod validation, so even a successful API call with malformed output triggers fallback. Third, I engineered the prompt carefully with 10 grounding rules telling the model to never invent data, never assume causation, and return only structured JSON. Plus a server-side safeguard: if there are zero clicks, I override whatever the AI says with a hardcoded 'Insufficient Data' insight."

---

20. HOW TO EXPLAIN MY CONTRIBUTION

Based on the git history, **Karthik M R** (the repository owner) appears to be the **sole contributor**. The commit history shows a clear progression:

1. `08dcf25` — Initial setup (landing page)
2. `d73bf5c` — Authentication (frontend + backend)
3. `17ed58d` → `6bd22a3` — Project CRUD
4. `ace7bc3` → `4abf39d` — Campaign module
5. `6c23664` → `6a60a10` — Redirect engine + UUID visitor tracking
6. `ec5d0b3` → `3f7e27c` — Analytics backend + frontend
7. `fc8f9c7` — AI integration with fallback
8. `88fca1e` → multiple — Deployment fixes (CORS, cookies, SPA routing)
9. `d43964a` — AI hallucination fix
10. `0281f9b` — Referrer fix

Interview Answer: "I designed and built every part of LaunchLens — from the database schema design in Prisma to the redirect engine, from the analytics aggregation service to the multi-provider AI layer, from the React component architecture to the deployment configuration on Vercel and Render. The biggest technical challenges were: getting cross-origin cookies working between Vercel and Render (required dynamic SameSite/Secure settings), fixing AI hallucination (turned out to be escaped template literals in my prompt builder), and designing the visitor tracking system to balance accuracy with simplicity."

---

21. CHALLENGES, TRADE-OFFS, AND ENGINEERING DECISIONS

Challenge 1: Cross-Origin Cookie Persistence

Problem: After deployment, login sessions didn't persist — cookies weren't sent cross-origin. **Current Solution:** Dynamic cookie config — `sameSite: 'none'` + `secure: true` in production. **Why This Approach:** Vercel and Render are on different domains. `SameSite: lax` blocks cross-origin cookie delivery. **Trade-off:** `SameSite: none` weakens CSRF protection. No CSRF token compensates for this. **Better Production Solution:** Use a custom domain so frontend and backend share the same parent domain (e.g., app.launchlens.com + api.launchlens.com), enabling `SameSite: lax`.

Challenge 2: AI Hallucination

Problem: AI was generating fictional campaign data instead of analyzing actual metrics. **Current Solution:** Fixed escaped template literals; added grounding rules; added zero-click override. **Alternative:** Could use function calling / structured output mode to give the AI even less room to deviate. **Limitation:** Prompt-based grounding is soft enforcement — the AI could still hallucinate with unusual data patterns.

Challenge 3: Analytics Performance

Problem: All analytics computed on-the-fly from raw events. **Current Solution:** `Promise.all()` for parallel queries, in-memory grouping. **Trade-off:** Simple to implement but O(n) in click events per query. Acceptable for MVP scale. **Better Production Solution:** Database-level `GROUP BY`, materialized views, Redis caching, background pre-aggregation.

Challenge 4: Visitor Identity

Problem: Need to distinguish unique visitors without requiring accounts. **Current Solution:** Persistent `ll_visitor` cookie with UUID. **Trade-off:** Identifies browsers, not humans. Multiple browsers / incognito = multiple visitors. **Better Production Solution:** Combine cookie + IP + User-Agent fingerprinting, or use authenticated visitor IDs for logged-in click recipients.

Challenge 5: SPA Routing on Vercel

Problem: Direct navigation to `/dashboard` returned 404 on Vercel. **Current Solution:** `vercel.json` with catch-all rewrite to `index.html`. **Why:** Vercel serves static files; only `index.html` exists. React Router needs to handle routes client-side.

---

22. CURRENT LIMITATIONS AND FUTURE IMPROVEMENTS

Easy Improvements

1. **Add Helmet middleware** — one line: `app.use(helmet())`
2. **Add rate limiting** — `express-rate-limit` on login and API endpoints
3. **Fix password validation mismatch** — backend requires 8 chars, frontend only 6
4. **Remove unused TanStack Query dependency** — installed but unused
5. **Implement the "Forgot Password" flow** — button exists but is non-functional
6. **Populate country field** — add a geo-IP lookup service (e.g., ipinfo.io)

Medium Engineering Improvements

1. **Database-level GROUP BY** for analytics instead of in-memory grouping
2. **Redis caching** for analytics with TTL-based invalidation
3. **Refresh token rotation** — short-lived access tokens + long-lived refresh tokens
4. **Error monitoring** — Sentry or similar for production error tracking
5. **API versioning** — prefix routes with `/api/v1/`
6. **Use the Conversion model** — implement the conversion tracking already in the schema
7. **Populate UTM fields** (source, medium, campaignTag) already in Campaign schema
8. **Bot detection** — filter common bot User-Agents from analytics

Production-Scale Improvements

1. **Async click ingestion** — Redis/Kafka queue for click events, background workers for database writes
2. **Pre-aggregated analytics** — background jobs computing daily/hourly summaries
3. **Read replicas** — separate analytics reads from write workload
4. **Table partitioning** — partition click_events by month/campaign
5. **CDN for tracking endpoint** — edge functions for minimal latency redirect
6. **Cost-aware AI routing** — track per-provider costs and prefer cheaper providers when quality is similar
7. **Custom domains** — let users use their own domain for tracking links
8. **Team workspaces with RBAC** — multi-user access to shared projects

---

23. INTERVIEW CHEAT SHEETS

A. 30-second pitch

"LaunchLens is a full-stack campaign analytics platform I built with React, Express, PostgreSQL, and a multi-provider AI layer. Users create campaigns with tracking links; clicks capture metadata and get recorded before a 302 redirect. The analytics service computes deterministic metrics, and the AI interprets them using Gemini/Groq/Mistral with automatic fallback. Everything is deployed on Vercel, Render, and Supabase."

B. 1-minute pitch

(See Section 2C)

C. Complete architecture in one page

```
React (Vite/TS) → Axios → Vercel Rewrites → Express (Node/TS) → Prisma → PostgreSQL (Supabase)
                                                  |
                                    ┌─────────────┼─────────────┐
                                    |             |             |
                                 Auth/JWT     Analytics      AI Layer
                                (bcrypt,      (queries on    (Gemini→Groq→Mistral)
                                 HTTP-only    click_events,  contextBuilder,
                                 cookies)     Promise.all)   promptBuilder,
                                                             providerRouter)
```

D. Complete user flow in one page

```
Landing → Signup → Login → Cookie Set → Dashboard (stats) →
Create Project → Create Campaign → Get Tracking Link →
Share Link → Visitor Clicks → Metadata Captured → ClickEvent Stored → 302 Redirect →
View Analytics (charts) → Generate AI Insights → Summary + Insights + Recommendations
```

E. Most important API endpoints

1. `POST /api/auth/login` — creates JWT cookie
2. `POST /api/projects/:pid/campaigns` — creates campaign + tracking link
3. `GET /r/:publicSlug` — tracking + redirect (core pipeline)
4. `GET /api/campaigns/:id/analytics` — campaign analytics
5. `POST /api/campaigns/:id/insights` — AI insights

F. Database models

`User → Project → Campaign → ClickEvent → Conversion (unused)`

G. Authentication flow

`Login → bcrypt compare → JWT sign → HTTP-only cookie → authenticate middleware → req.user.userId`

H. Tracking flow

`GET /r/:slug → findCampaign → getVisitorId(cookie) → detectDevice(UA) → parseBrowser(UAParser) → readReferrer → createClickEvent → redirect(302)`

I. Analytics flow

`getCampaignAnalytics → Promise.all(7 queries) → { campaign, summary, timeline, devices, browsers, countries, referrers }`

J. AI fallback flow

`buildContext → buildPrompts → Gemini(try) → Groq(try) → Mistral(try) → JSON.parse → Zod validate → grounding safeguard → return`

K. Top 30 questions I absolutely must know

1. Tell me about LaunchLens
2. What happens when a tracking link is clicked?
3. How do you identify unique visitors?
4. Why HTTP-only cookies?
5. How does the AI fallback work?
6. What is deterministic analytics?
7. Why PostgreSQL?
8. How do you prevent User A from seeing User B's data?
9. What security measures exist?
10. What security measures are missing?
11. Why 302 and not 301?
12. How does Prisma help?
13. What happens if the database write fails during redirect?

14. How does CORS work in your project?
15. What deployment challenges did you face?
16. How would you scale to 1M clicks/day?
17. What does the Zod validation do?
18. How does the auth middleware work?
19. What is the session restoration flow?
20. What would you improve first?
21. How do you handle AI hallucination?
22. What's in the JWT payload?
23. Why is the visitor cookie not `secure`?
24. How are analytics calculated?
25. What edge cases exist in referrer tracking?
26. Why use Vercel rewrites?
27. What's the difference between authentication and authorization?
28. Why three AI providers?
29. What data does the AI receive?
30. What is the Conversion model for?

L. Top 20 technical terms I must explain confidently

1. JWT (JSON Web Token) — stateless auth token with payload + signature
2. HTTP-only cookie — cookie inaccessible to JavaScript
3. bcrypt — adaptive password hashing algorithm with salt
4. CORS — browser policy controlling cross-origin requests
5. SameSite cookie attribute — controls when cookies are sent cross-site
6. 302 redirect — temporary redirect (browser doesn't cache)
7. User-Agent — HTTP header identifying browser/OS
8. ORM (Object-Relational Mapping) — maps code objects to database tables
9. Prisma — type-safe Node.js ORM with schema-first approach
10. Zod — runtime data validation library for JavaScript/TypeScript
11. UUID — universally unique identifier (128-bit)
12. Slug — URL-friendly string derived from a name
13. XSS — cross-site scripting (inject malicious JavaScript)
14. CSRF — cross-site request forgery (trick browser into making requests)
15. Rate limiting — restricting request frequency per client
16. Provider fallback — trying backup services when primary fails
17. Deterministic — produces same result given same inputs (not probabilistic)
18. Grounding — constraining AI output to factual, provided data
19. Connection pooling — reusing database connections (PgBouncer)
20. SPA (Single Page Application) — one HTML page, client-side routing

M. Top 10 challenges and improvements

1. Cross-origin cookies → Dynamic SameSite config → Custom domain sharing
2. AI hallucination → Template fix + grounding rules → Function calling
3. Analytics performance → In-memory grouping → Database GROUP BY + caching
4. Visitor identity → Cookie UUID → Multi-signal fingerprinting
5. No rate limiting → None → express-rate-limit + Redis
6. No bot detection → None → User-Agent filtering + heuristics
7. Synchronous click storage → Blocking redirect → Message queue
8. No security headers → None → Helmet middleware
9. No refresh tokens → 7-day JWT → Access/refresh rotation
10. Country data missing → Null → Geo-IP lookup service

---

24. IMPLEMENTATION VERIFICATION REPORT

Feature / Resume Claim	Status	Evidence
Tracking and redirect pipeline	✅ VERIFIED	`redirect.controller.ts` — GET /r/:publicSlug, ClickEvent creation, 302 redirect
Browser detection with UAParser	✅ VERIFIED	`redirect.controller.ts` — `new UAParser(userAgentString).getBrowser().name`
Device detection	✅ VERIFIED	`detectDevice.ts` — string matching on User-Agent
Referrer tracking	✅ VERIFIED	`redirect.controller.ts` — `req.get("referer")`, `getReferrerAnalytics()` hostname extraction

Feature / Resume Claim	Status	Evidence
Timestamp metadata	✅ VERIFIED	`ClickEvent.clickedAt` — `@default(now())` in schema
Unique visitor tracking (cookie UUID)	✅ VERIFIED	`getVisitorId.ts` — `crypto.randomUUID()`, `ll_visitor` cookie
PostgreSQL analytics	✅ VERIFIED	`analytics.service.ts` — all metrics from PostgreSQL via Prisma
Timeline analytics	✅ VERIFIED	`getTimelineAnalytics()` — date-grouped click counts
Device analytics	✅ VERIFIED	`getDeviceAnalytics()` — device type distribution
Browser analytics	✅ VERIFIED	`getBrowserAnalytics()` — browser name distribution
Traffic source analytics	✅ VERIFIED	`getReferrerAnalytics()` — referrer hostname distribution
Country analytics	⚠️ PARTIALLY VERIFIED	Schema has `country` field; `getCountryAnalytics()` exists; BUT `country` is **never populated** in redirect controller
Prisma ORM	✅ VERIFIED	`schema.prisma`, singleton client, used in every backend file
JWT authentication	✅ VERIFIED	`jwt.ts` — sign/verify with 7-day expiry
HTTP-only cookies	✅ VERIFIED	`auth.controller.ts` — `httpOnly: true`
bcrypt password hashing	✅ VERIFIED	`hash.ts` — bcrypt with 10 salt rounds
Protected APIs	✅ VERIFIED	`authenticate` middleware on all protected routes
CORS configuration	✅ VERIFIED	`app.ts` — explicit origin allowlist with credentials
Input validation (Zod)	✅ VERIFIED	Zod schemas in `validation/` for auth, project, campaign, AI
User-level ownership checks	✅ VERIFIED	Every controller filters by `req.user.userId`
Gemini AI integration	✅ VERIFIED	`gemini.provider.ts` — @google/genai SDK, gemini-2.5-flash
Groq AI integration	✅ VERIFIED	`groq.provider.ts` — raw fetch, llama-3.3-70b-versatile
Mistral AI integration	✅ VERIFIED	`mistral.provider.ts` — raw fetch, mistral-small-latest
Multi-provider fallback	✅ VERIFIED	`providerRouter.ts` — sequential try/catch through 3 providers
Deterministic analytics before AI	✅ VERIFIED	`aiInsight.service.ts` → `getCampaignAnalytics()` → `buildAIContext()`
AI grounding safeguard	✅ VERIFIED	`aiInsight.service.ts` lines 110-120 — zero-click override
AI response Zod validation	✅ VERIFIED	`aiInsightSchema` in `ai.validation.ts`, validated in both router and service
Vercel frontend deployment	✅ VERIFIED	`vercel.json` with rewrites
Render backend deployment	✅ VERIFIED	Render URL in `vercel.json` rewrite destination
Supabase PostgreSQL	✅ VERIFIED	Supabase connection string in `.env`
Conversion tracking	❌ NOT IMPLEMENTED	Schema model exists; no controller/service uses it
Campaign UTM fields (source, medium, tag)	❌ NOT IMPLEMENTED	Schema fields exist; never populated by any controller
Rate limiting	❌ NOT IMPLEMENTED	Not present in codebase
Helmet / security headers	❌ NOT IMPLEMENTED	Not present in codebase
CSRF protection	❌ NOT IMPLEMENTED	No CSRF tokens
Email verification	❌ NOT IMPLEMENTED	Not present in codebase
Forgot password	❌ NOT IMPLEMENTED	Button exists in UI but is non-functional
TanStack Query usage	❌ NOT USED	Installed as dependency but no component uses it
Geo-IP country lookup	❌ NOT IMPLEMENTED	Country field exists but is never populated